# 2010376103 CSE4261 Assignment-12

OMOR NILOY

July 2025

## Dataset Preparation

In all experiments, the **CIFAR-10** dataset was used. It contains 60,000 color images of size $32 \times 32$ pixels, divided equally among 10 different classes. To ensure experimental reproducibility, a fixed random seed (42) was set across all random number generators: `torch`, `numpy`, and `random`. To reduce computational overhead and speed up experimentation, only a subset of 10,000 samples was randomly selected from the original 50,000 training images. This reduced subset was split into training and validation sets using a 90%–10% split ratio, with randomness controlled by a seeded generator. Three datasets were prepared:

- **Training set:** This set underwent strong data augmentation to improve model generalization. The applied transformations include:
  - Random resized crop
  - Random horizontal flip
  - Random rotation within $[-30°, +30°]$
  - `RandAugment` with 2 operations and magnitude 7
  - Normalization using ImageNet mean and standard deviation
- **Validation and Test sets:** No augmentation was applied to these sets. Only normalization was performed to maintain consistency and allow fair evaluation.

All datasets were loaded using PyTorch's `DataLoader` with a batch size of 64. Data loading was parallelized using all available CPU cores via `os.cpu_count()` to maximize efficiency.

This controlled data preparation pipeline ensured consistency across all training regimes and allowed for reliable comparison of model performances.

# 1 Building a small CNN based classifier, and training it by your favorite dataset (except the MNIST digit or fashion dataset) having images of 10 classes.

Github code link

## 1.1 Small CNN model

To fulfill the requirement of building a custom image classifier, we developed a lightweight Convolutional Neural Network (CNN) and trained it on the CIFAR-10 dataset.

The architecture of our CNN model was kept simple yet effective, suitable for low-resource environments. It includes two convolutional blocks followed by pooling, then fully connected layers for classification. Below is a brief summary of the model architecture:

- **Input:** RGB images of size $32 \times 32 \times 3$
- **Conv Block 1:** Conv2D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPooling
- **Conv Block 2:** Conv2D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPooling
- **Flatten Layer**
- **Fully Connected Layers:** Dense $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Output Layer
- **Output:** 10-class softmax

### 1.2 Training Setup

This CNN was trained from scratch using the training set, with standard data augmentation techniques applied to improve generalization. The training process 50 epochs used CrossEntropy loss, the Adam optimizer, and a learning rate scheduler. Validation accuracy and loss were tracked to ensure effective training without overfitting.

### 1.3 Results of Small CNN

The small CNN model was trained for 50 epochs on the CIFAR-10 dataset using a reduced subset of 10,000 training images and validated on a held-out validation set. After training, the model achieved the following performance on the test set:

- **Test Accuracy:** 71.83%
- **Test Loss:** 1.1520

Figure 1 shows the training and validation accuracy and loss curves across epochs. As observed, the model converged steadily with a gap between training and validation performance.

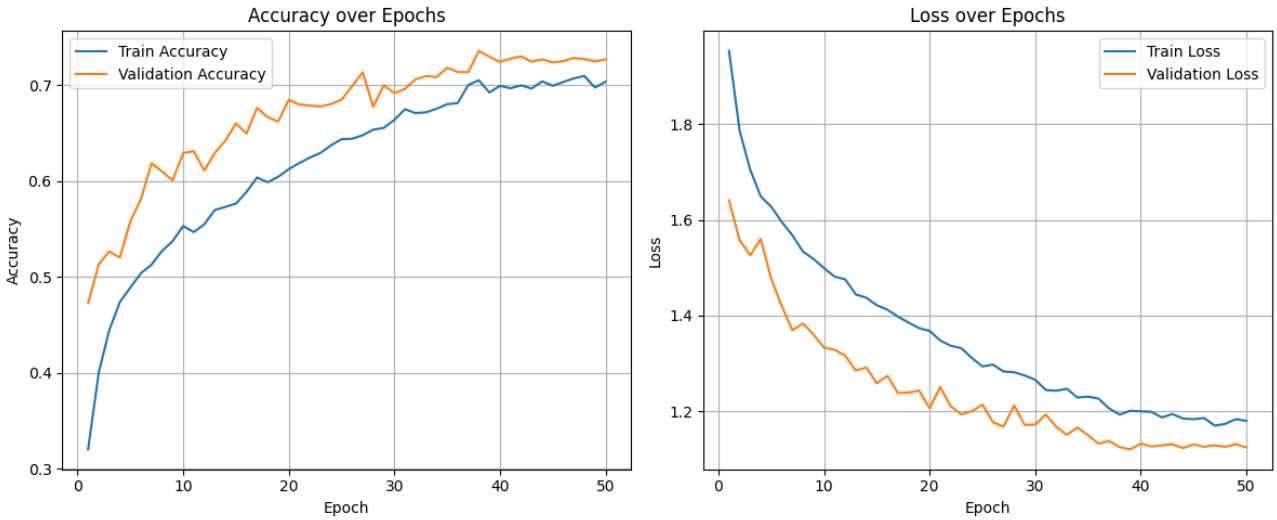


Figure 1: Training and Validation Accuracy/Loss of the Small CNN model over epochs.

## 2 Preparing your two favorite big CNNs, pre-trained by ImageNet dataset for 1000 classes, for classifying 10 classes of your chosen dataset by fine tuning last 1/2 layers.

Github code link

### 2.1 Fine-Tuning Pretrained CNNs

To improve performance over the small CNN baseline, two well-known convolutional neural network architectures pretrained on the ImageNet dataset were utilized:

- **ResNet50**
- **MobileNetV3-Large**

Both models were initialized with ImageNet weights and then fine-tuned on the CIFAR-10 dataset. Since ImageNet is a much larger dataset with 1000 classes, only the final layers of these models were modified and retrained to adapt to the 10-class classification task of CIFAR-10. Specifically, the following steps were taken:

1. Replaced the final classification layer with a new fully connected layer of size 10 to match CIFAR-10's classes.
2. Fine-tuned only the last half layers, while keeping the earlier layers frozen to preserve the learned features from ImageNet.

## 2.2 Training Setup

Both models were trained using the same reduced CIFAR-10 training set of 10,000 images, with validation and test splits fixed using a manual seed for reproducibility. The models were optimized using the Adam optimizer and trained with a cosine annealing learning rate scheduler for up to 50 epochs.

## 2.3 Results of Pretrained Models

The following performance was achieved on the CIFAR-10 test set after fine-tuning:

- **ResNet50**: **Test Accuracy: 84.04%**, **Test Loss: 0.9006**
- **MobileNetV3-Large**: **Test Accuracy: 83.93%**, **Test Loss: 0.9029**

Figure 2, 3 shows the training and validation accuracy and loss curves across epochs.

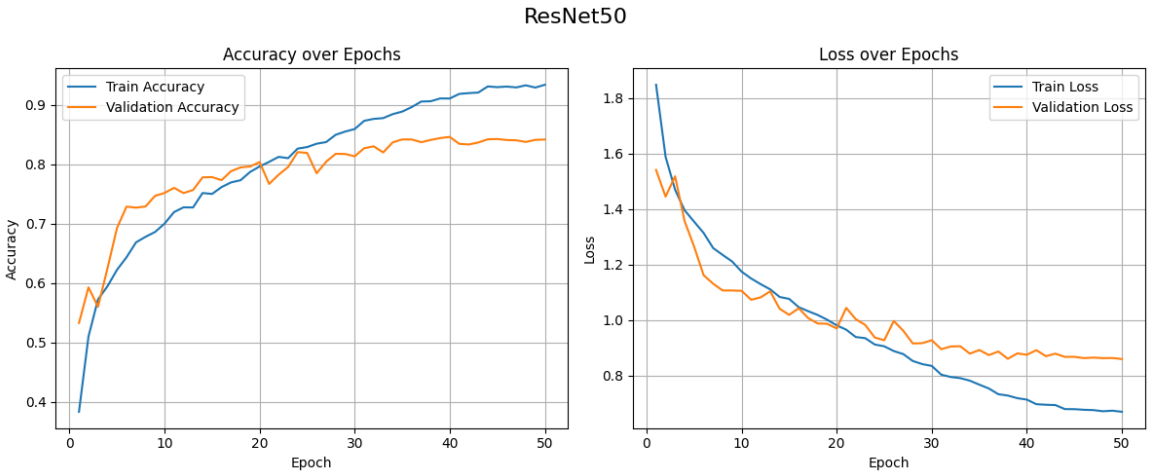


Figure 2: Training and Validation Accuracy/Loss for ResNet50

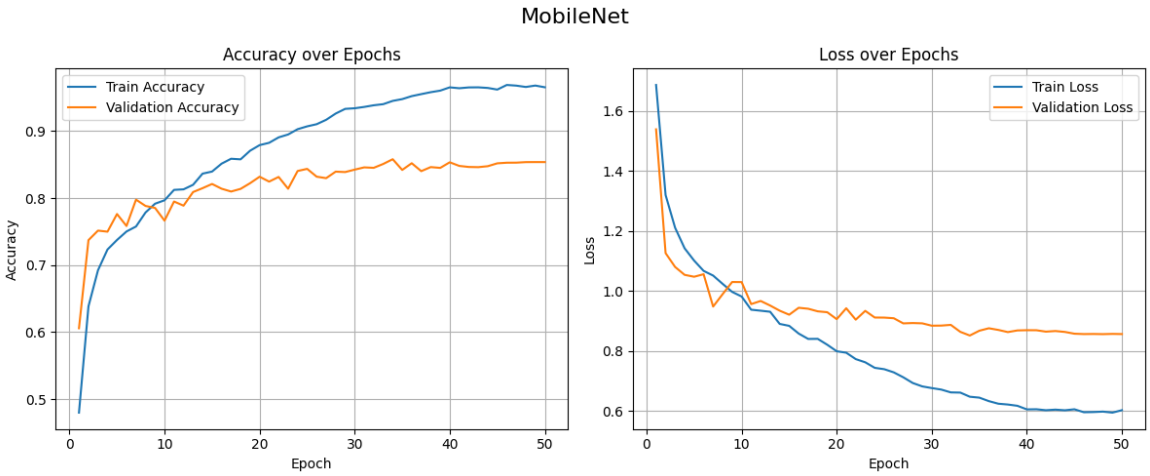


Figure 3: Training and Validation Accuracy/Loss for MobileNetV3-Large.

## Knowledge Distillation

Knowledge Distillation (KD) is a model compression technique that enables a smaller model (called the *student*) to learn from a larger, more powerful model (called the *teacher*). The core idea is to transfer the knowledge captured by the teacher into the student in order to improve its generalization performance without significantly increasing its complexity. Instead of training the student solely on the hard ground-truth labels, KD introduces a soft target derived from the teacher model's output probabilities. This is achieved by minimizing a combination of two losses:

- **Hard Target Loss** ($\mathcal{L}_{\text{CE}}$): Standard cross-entropy loss between the student's predictions and the ground truth labels.
- **Soft Target Loss** ($\mathcal{L}_{\text{KD}}$): Kullback–Leibler divergence (KL-divergence) between the softened output distributions (soft logits) of the teacher and student models.

The total loss function used for KD is given by:

$$\mathcal{L}_{\text{Total}} = (1 - \alpha) \cdot \mathcal{L}_{\text{CE}} + \alpha \cdot T^2 \cdot \mathcal{L}_{\text{KD}}(\mathbf{z}_s/T, \mathbf{z}_t/T)$$

where:

- $\mathbf{z}_s$ and $\mathbf{z}_t$ are the logits of the student and teacher, respectively,
- $T$ is the temperature parameter used to soften the logits,
- $\alpha \in [0, 1]$ controls the trade-off between the two losses.

This formulation encourages the student to not only predict the correct class, but also mimic the teacher's confidence distribution over all classes, which provides richer supervision.

# 3 Transferring knowledge from your big fine-tuned one classifier to your own small CNN

Github code link

## 3.1 Training Setup

In this experiment, we aimed to improve the performance of our custom small CNN model by transferring knowledge from a larger, more powerful model—ResNet50—which was fine-tuned on our dataset. First, we fine-tuned a pre-trained ResNet50 (originally trained on ImageNet) on our 10-class dataset for 30 epochs. Afterward, we used this trained model as a teacher for knowledge distillation. The knowledge distillation process involved training the small CNN (student) using a combination of two losses:

- **Soft target loss** using the Kullback–Leibler (KL) divergence between the teacher's soft logits and the student's output.
- **Hard label loss** using cross-entropy with ground-truth labels.

The final loss function is a weighted sum:

$$\mathcal{L}_{\text{KD}} = (1 - \alpha) \cdot \mathcal{L}_{\text{CE}} + \alpha \cdot T^2 \cdot \mathcal{L}_{\text{KL}}$$

where $\alpha = 0.8$ is the balancing factor, $T = 4$ is the temperature, $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss, and $\mathcal{L}_{\text{KL}}$ is the KL divergence loss between soft logits.

## 3.2 Results of ResNet50 as Teacher

We compare the performance of three models:

- **Fine-tuned ResNet50 (Teacher)**: Validation Accuracy = **83.54%**, Loss = **0.8976**
- **Small CNN (Trained from Scratch)**: Validation Accuracy = **69.09%**, Loss = **1.1999**
- **Small CNN (Trained with Knowledge Distillation)**: Validation Accuracy = **72.88%**, Loss = **1.1367**

The knowledge distillation method improves accuracy from scratch training. Figure 4, 5, 6 shows the training and validation accuracy and loss curves across epochs.

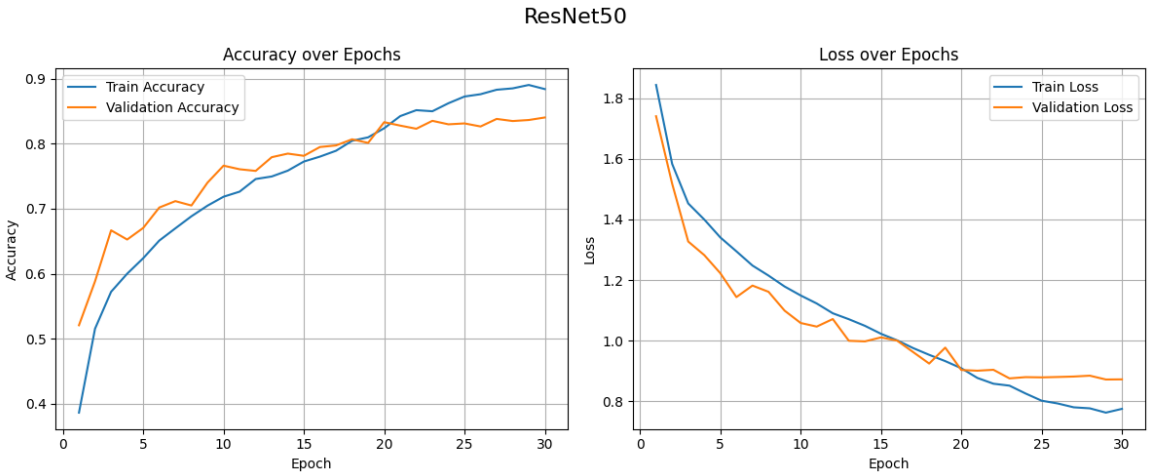


Figure 4: Training and Validation Accuracy/Loss for Resnet50.

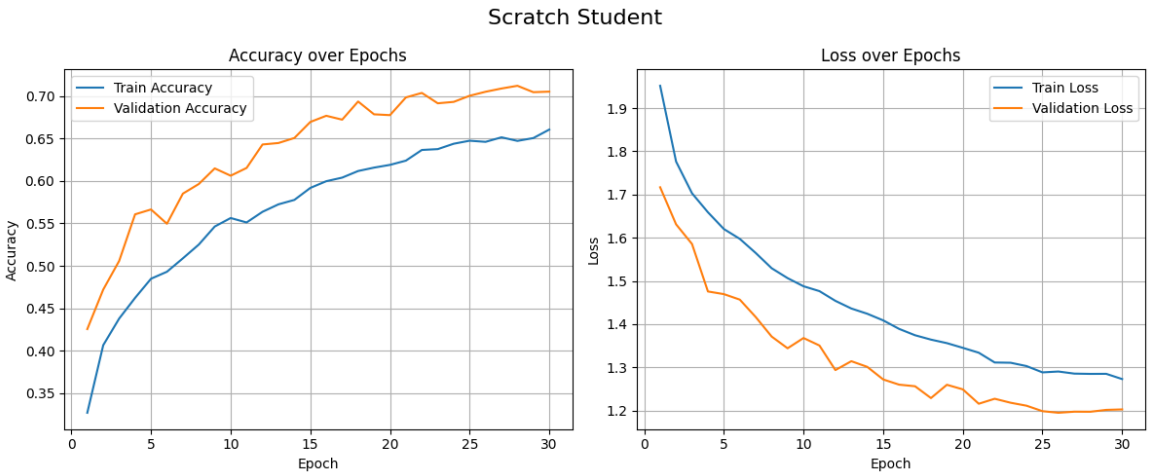


Figure 5: Training and Validation Accuracy/Loss for Scratch Student.

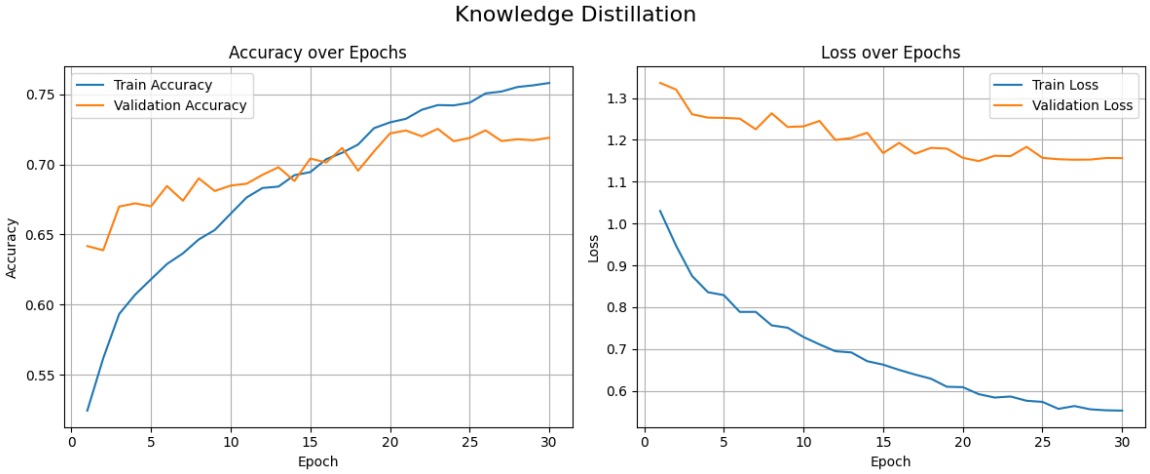


Figure 6: Training and Validation Accuracy/Loss for Knowledge Distillation.

# 4 Transferring knowledge from your big fine-tuned two classifiers to your own small CNN

Github code link

## 4.1 Training Setup

In this experiment, we explore knowledge distillation using a different large model—MobileNetV3-Large—as the teacher. Similar to the previous setup, the MobileNetV3 model was first fine-tuned on our 10-class dataset. After 30 epochs of training, it served as the teacher model for distilling knowledge into our small CNN (student) model. We applied the same knowledge distillation strategy, where the student is trained with a loss function combining both:

- **Soft target loss**: KL divergence between the teacher's softened logits and the student's outputs.
- **Hard label loss**: Cross-entropy loss using the ground-truth labels.

The total loss is:

$$\mathcal{L}_{\text{KD}} = (1 - \alpha) \cdot \mathcal{L}_{\text{CE}} + \alpha \cdot T^2 \cdot \mathcal{L}_{\text{KL}}$$

where we used $\alpha = 0.8$ and $T = 4$, consistent with the earlier setup.

## 4.2 Results of MobileNet as Teacher

Below are the validation performances of the models involved:

- **Fine-tuned MobileNetV3 (Teacher)**: Validation Accuracy = **83.63%**, Loss = **0.9026**
- **Small CNN (Trained from Scratch)**: Validation Accuracy = **69.98%**, Loss = **1.1810**
- **Small CNN (Trained with Knowledge Distillation)**: Validation Accuracy = **72.35%**, Loss = **1.1643**

The knowledge distillation method improves accuracy from scratch training. Figure 7, 8, 9 show the training and validation metrics across epochs for all three configurations.

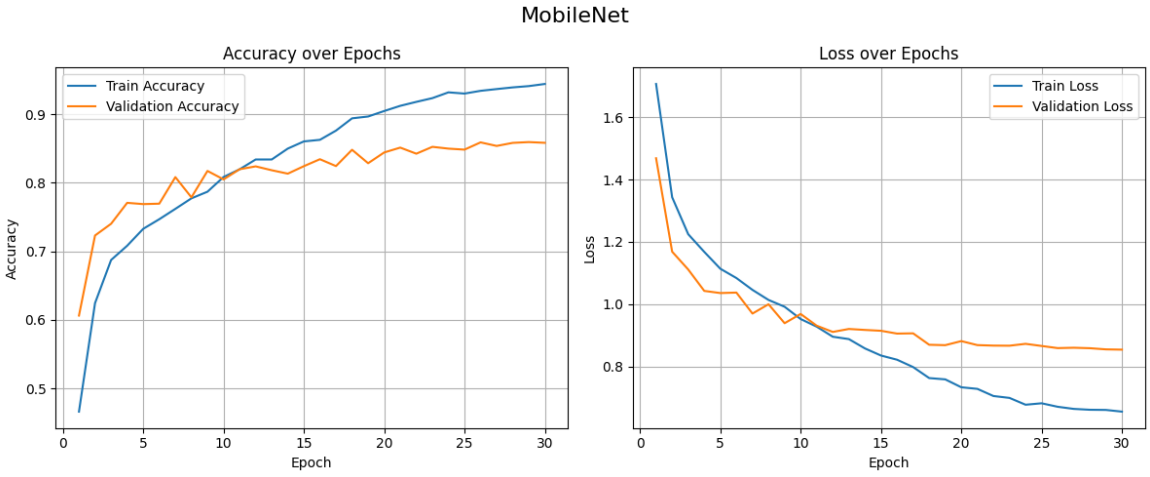


Figure 7: Training and Validation Accuracy/Loss for MobileNetV3-Large.

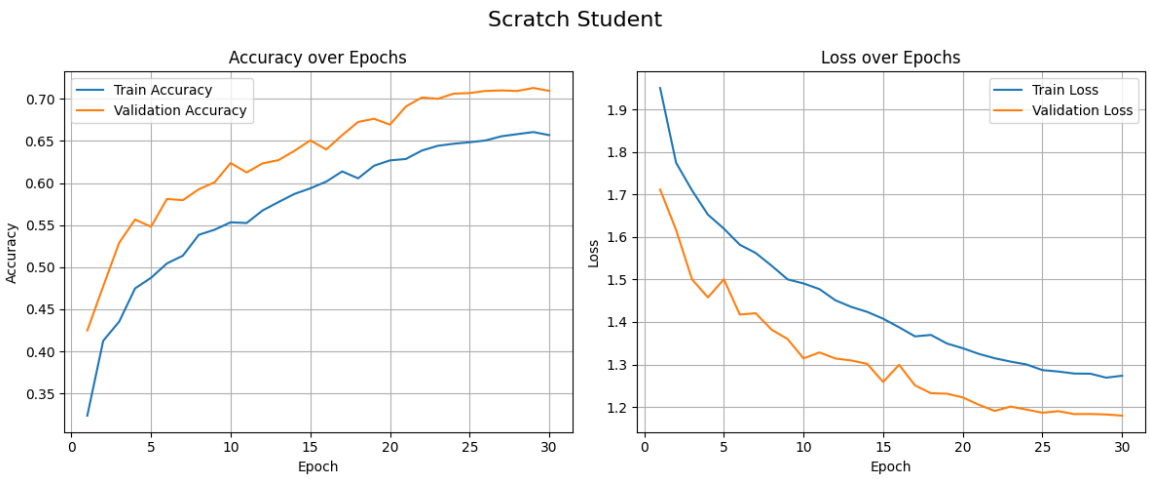


Figure 8: Training and Validation Accuracy/Loss for Scratch Student.

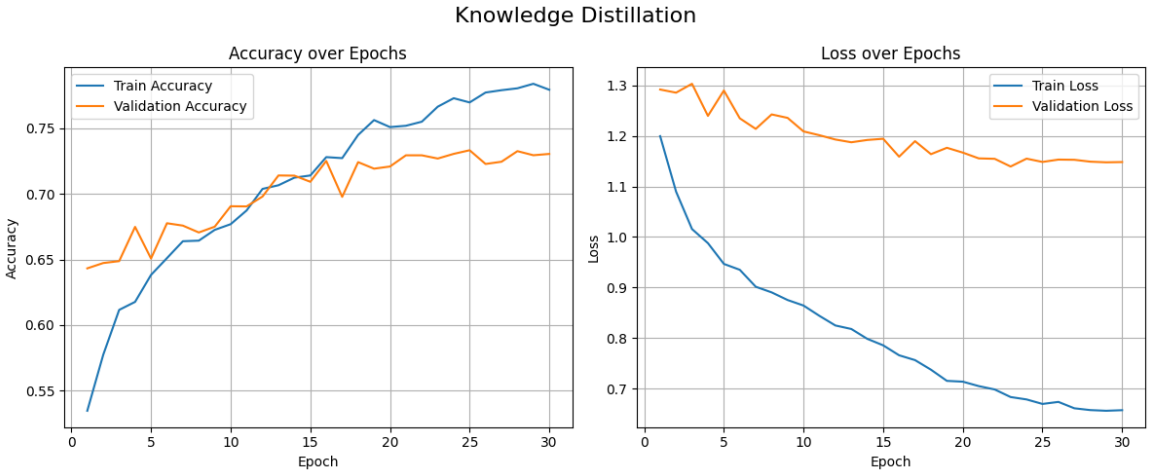


Figure 9: Training and Validation Accuracy/Loss for Knowledge Distillation.